10.9.26 LMB HY 2023-24

a) NOR (iv) ✓

b) XNOR (i) ✓

c) Contradiction (iii) ✓

d) OR (ii) ✓

e) new (iv) ✗ this (i)

f) Propositions are also called well formed sentences, statements or formula. That is, well formed formula (wff) refers to a proposition. ✓

g) $(AB + C'D')'$

~~A B.~~

$= (AB)' \cdot (C'D')'$

$= (\bar{A} + \bar{B})(C + D)$ ✓

~~A C + A~~

h) $(A+B) \cdot B' \cdot 0 = 0$ ✓

i) $(A \rightarrow B)$'s contrapositive form is $(\bar{B} \rightarrow \bar{A})$. ✓

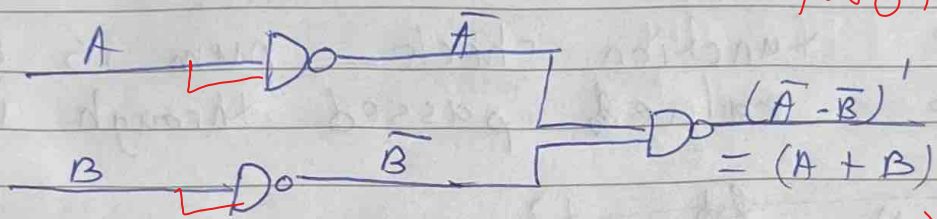
j)

A	B	$f(A,B) = A \cdot B$
0	0	0
0	1	0
1	0	0
1	1	1

$\therefore POS = (A+B)(\bar{A}+B)(A+\bar{B})$ ✓

NAND gate must have two inputs

2) a)



✓

b) De-Morgan's theorum states that if the conjuction of A and B ~~(A+B)~~ is complemented, it will result in the disjunction of the complements of A and B, that is, $(A+B)' = A' \cdot B'$, and if the disjunction of A and B is complemented, it will result in the conjuction of the complements of A and B, that is, $(A \cdot B)' = A' + B'$.

✓

c) i) a | a | b | c | s | d

3243	0	3243	324	0	3
	3	324	32	0	4
	34	32	3	0	2
	342	3	30	00	3
	3423	0			

∴ Value returned = 3243 ✓

ii) The function check_num is reversing the values passed through it. ✓

d) n → t1 → t2

n	t1	t2	Output
449	0	2	t2 = 2
2095	2	2	t2 = 2
134	4	4	t2 = 4
695	8	1	t2 = 1
46	9		
3			
1.5			
1			

∴ Final Output is as follows: -

t2 = 2
t2 = 2
t2 = 4
t2 = 1
t1 = 9 ✓

3) a) $(110101.110)_2$

$= 1\times2^5 + 1\times2^4 + 0\times2^3 + 1\times2^2 + 0\times2^1 + 1\times2^0 + 1\times2^{-1} + 1\times2^{-2} + 0\times2^{-3}$

$= (53.75)_{10}$ ✓

b) $(ABI9E)_{16}$

~~$\rightarrow 1010111001$~~

$= (\overline{0}\,\overline{1}\,\overline{0}\,\overline{1}\,\overline{0}\,\overline{1}\,\overline{1}\,\overline{0}\,\overline{0}\,\overline{0}\,\overline{1}\,\overline{1}\,\overline{0}\,\overline{0}\,\overline{1}\,\overline{1}\,\overline{1}\,\overline{1}\,\overline{0})_2$ mark the groupings

$= (2530636)_8$ ✓

c) $(76523)_{10}$

2 | 76523		
2 | 38261	1	
2 | 19130	1	
2 | 9565	0	
2 | 4782	1	
2 | 2391	1	
2 | 1195	1	
2 | 597	1	
2 | 298	1	
2 | 149	0	

16 | 76523		
16 | 4782	B	
16 | 298	E	
16 | 18	A	
16 | 1	2	
	0	1

$\therefore (76523)_{10} = (12AEB)_{16}$ ✓

d) $(110111)_2 + (10111)_2$

∴ 1's complement of $(10111)_2 = (01000)_2$

∴ 2's complement of $(10111)_2 = 1000 + 1 = (1001)_2$

∴ Sum of $(110111)_2$ & and 2's complement of $(10111)_2$ = 110111 001

∴
$$\begin{array}{r} 110111 \\ +\ 10111 \\ \hline 1001110 \end{array}$$

∴ Ans = $(1001110)_2$ ✓

e) $(11100101)_2 - (11011)_2$

∴ 1's complement of $(11011)_2 = (00100)_2$

∴ 2's complement of $(11011)_2 = 100 + 1 = (101)_2$

∴ Adding $(11100101)_2$ to 2's complement of $(11011)_2$ = 11100101 + 101 = 11101010

∴ Ans = $(11101010)_2$

∴ 1's complement of $(00011011)_2 = (11100100)_2$

∴ 2's complement of $(00011011)_2$ = 11100100
+1
$(11100101)_2$ ✓

∴ Adding $(11100101)_2$ to 2's complement of $(00011011)_2$ = 11100101
+ 11100101
$(1)11001010)_2$

∴ Discarding extra digit = $(11001010)_2$ ✓

∴ Ans = $(11001010)_2$

4|a) G C E X

0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

∴ Terms with conjunction for each of the true values = $(\bar{G}+\bar{C}+\bar{E})$, $(\bar{G}+C+E)$, $(G+\bar{C}+E)$, $(G+C+\bar{E})$, $(G+C+E)$

∴ Propositional expression = $(\bar{G}+\bar{C}+\bar{E}).(\bar{G}+C+E).(G+\bar{C}+\bar{E}).(G+C+\bar{E}).(G+C+E)$ ✓

$X(G, C, E) = T$

A combinational ckt. which adds 2-bit and produces sum and carry as output. It uses XOR gate & AND gate.

b) A combinational circuit with 2 binary bits and uses a XOR and AND gate is called Half adder.

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$\therefore Sum = \bar{A}B + A\bar{B} + \bar{A}B +$

$\therefore Sum = \bar{A}\bar{B} + \bar{A}B + A\bar{B} + AB$

$= \bar{A}(B + \bar{B})$

$\therefore Sum = \bar{A}B + A\bar{B}$

$\therefore Carry = AB$

c) $M = XY + X(Y+Z) + Y(Y+Z)$

$= XY + (Y+Z)(X+Y)$

$= XY + (Y + XZ)$ (converse of By Distributive Law)

$= XY + Y + XZ$

$= X(X+1) + XZ$

$= X + XZ$

$= (X+X)(Y+Z)$

Sol) The difference between half adder and full adder is :-

Half- Adder	Full- Adder
• Operates using 2- Binary Bits	• Operates using 3- Binary Bits

A	B	C	Sum	Carry
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$\therefore$ Sum $= \bar{A}\bar{B}C + \bar{A}B\bar{C} + A\bar{B}\bar{C} + ABC$

$\therefore$ Carry $= \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC$

$= \bar{A}BC + A\bar{B}C + AB + ABC$

$= \bar{A}BC + A(B + \bar{B}C)$

$= \bar{A}BC + A(B + C)$

$= \bar{A}BC + AB + AC$

$= B(\bar{A}C + A) + AC$

$= B(A + C) + AC$

Carry $= AB + BC + AC$

b) $[(a \Rightarrow b) \wedge (b \Rightarrow c)] \Rightarrow (a \Rightarrow c)$

~~$\bar{a} + b$~~

a	b	c	$a \Rightarrow b$	$b \Rightarrow c$	$(a \Rightarrow b) \wedge (b \Rightarrow c)$
0	0	0	1	1	1
0	0	1	1	1	1
0	1	0	1	0	0
0	1	1	1	1	1
1	0	0	0	1	0
1	0	1	0	1	0
1	1	0	1	0	0
1	1	1	1	1	1

$a \Rightarrow c$	$[(a \Rightarrow b) \wedge (b \Rightarrow c)] \Rightarrow (a \Rightarrow c)$
1	1
1	1
1	1
1	1
0	1
1	1
0	1
1	1

∴ The above expression is a tautology

c) The principle of duality states that any valid algebric expression or theorum remains valid if the AND and OR operators are swapped and the binary 0 and 1 are interchanged.

6)
```
import java.util.*

class Unique
{
 String n;

 Unique()
 {
   n= "";
 }
//Accepting the String
 void input()
 {
Scanner sc = new Scanner(System.in);
System.out.println("Enter the word");
n = sc.nextLine();
 }
//extracting the 1st and last character of a word and checking if they are the same
 boolean check(String b)
 {
  char a= b.charAt(0);
  char d= b.charAt(b.length()-1);
  a= Character.toUpperCase(a);
  d= Character.toUpperCase(d);
```

```
        if (a==d)
        return true;
        else
        return false;
    }
```

→ return (a==d);

```
// Printing the unique words
    void display()
    {
    String[] m = n.split(" ");
     System.out.println("Unique words are: ");
    for (int i=0; i<m.length; i++)
     {
      boolean k= check(m[i]);

      if (k==true)
      System.out.println(m[i]);
     }
}

public static void main()
 {
 Unique ob = new Unique();
      ob.input();
      ob.display();
 }
```

7)

```
import java.util.*;

class ColSum
 {
  int a[][];
  int row, col;
```

```
ColSum(int x, int y)
  {
     row = x;
     col = y

  for (int i=0 ; i<row ; i++)
     {
    for (int j=0; j<col ; j++)
     {
        a[i][j] = 0;
     }
     }
  }
// Accepting the Matrix
   void input()
     {
  Scanner sc = new Scanner(System.in);
  System.out.println("Enter the array");

    for (int i = 0; i<row ; i++)
     {
    for (int j=0; j<col; j++)
     {
        a[i][j] = sc.nextInt();
     }
     }
  }
// Calculating sum of column 'c'
   int sum(int c)
     {
       int s = 0;
```

Not needed JVM by default initializes with '0'

```
if(c<col){
    for(int k=0 ; k<row ; k++)
    {
    s=a[k][c];
    }
    return s;
}
}
// Printing Matrix and the sum of each column
void display()
{
System.out.println("Entered Array:");

for(int i=0 ; i<row ; i++)
{
for(int j=0; j<col ; j++)
{
System.out.print(a[i][j]+"\t");
}
System.out.println();
}

for(int k=0 ; k<col ; k++)
{
int s=sum(k);
System.out.println("Sum of colum: "+(k+1)+
" is "+s);
}
}

public static void main()
{
Scanner sc=new Scanner(System.in);
```

always use

```
System.out.println("Enter the row and column length");
int r = sc.nextInt();
int c = sc.nextInt();

colSum ob = new colSum(r, c);

        ob.input();
        ob.display();
    }
}
```

8)

```
import java.util.*;

class sumPrime
{
int n, sum;

sumPrime(int nn)
  {
   n = nn;
   sum=0;
  }
//Accepting the number
 void readdata()
  {
 Scanner sc=new Scanner(System.in);
  System.out.println("Enter the number");
     n = sc.nextInt();
 }
```

```
// Calculating Sum of Square of each individual digit
int calc()
  {
    int s=0;
    int copy= n;

    while (copy!=0)
    {
    s+= (copy%10) * (copy%10) ;
      copy /=10;
    }
    return s;
  }
// Checking if the sum is prime or not
boolean prime (sum)
  {
     sum= calc();
     int c=0;
    for(int i=2; i<sum; i++)
    {
      if (sum% i ==0)
        c++;
    }

    if (c==0)
    return true;
      else
      return false;
  }
```

Math.Sqrt(sum)

return (c==0);

```
void display()
  {
    boolean b= prime();
    if (b==true)
    {
System.out.println("Sum of squares of individual
                    dig its is prime");
    }
    else
    {
System.out.println("Sum of squares of individual
                    digits is not prime");
    }
}

public static void main()
  {
  int a= 0;

  sumPrime ob= new sumPrime(a);

    ob.readdata();
    ob.display();
  }
}
```

9)
```
import java.util.*;

  class Fibonacci
    {
      int n;
```

```
// Initializing n with x
Fibonacci(int x)
{
    n = x;
}
// Generating the nth term of the Fibonacci Series
int generate(int x)
{
    int a = 0, b = 1;

    for (int i=3; i<=x; i++)
    {
        a += b;
        b += a;
    }

    if (x%2 == 0)
        return b;
    else
        return a;
}
// Printing the nth term of the Fibonacci Series
void display()
{
    int d = generate();

    System.out.println("Term " + n +
        " of fibonacci series" +
        d );
}
System.out.println("Term "+n+
    "th fibonacci number: "+
    generate(n));
}
```

```
int a=-1, b=1, f=0;
while(x>0){
    f = a+b;
    a = b;
    b = f;
}
return f;
```

```
public static void main()
{
Scanner sc = new Scanner (System.in);
System.out.println ("Enter the term ");
    int y = sc.nextInt();
Fibonacci ob = new Fibonacci(y);
    ob.display();
}
}
```

10)

```
import java.util.*;
class MatPalin
{
  int a[][];
  int m, n;
MatPalin (int mm, int nn)
{
  m = mm;
  n = nn;
for (int i=0; i<m; i++)
{
for (int j=0; j<n; j++)
{
  a[i][j] = 0;
}
}
}
```

not needed. reason written above.

Accepting the Matrix

```
void fill()
{
Scanner sc = new Scanner(System.in);
System.out.println("Enter the Matrix");

for(int i=0; i<m; i++)
  {
  for(int j=0; j<n; j++)
    {
    a[i][j] = sc.nextInt();
    }
  }
}
// Checking if the no. x is Palindrome or not
boolean Palin(int x)
  {
    int copy = x, r=0;

    while(copy!=0)
      {
      r+= copy%10;
       r*=10;
       copy/=10;
      }

    if(r==x){
    System.out.println("
       return true;
    }
  else {
    return false;
    }
}
```

return (r==x);

Palindrome

```
// Printing all the Palindrome no.s
void display()
  {
    boolean b= palin

  System.out.println("The Palindrome numbers: ");

    for (int i= 0; i<m; i++)
      {
      for (int j=0; j<n; j++)
        {
          boolean b= Palin(a[i][j]);

          if (b==true)
        {
      System.out.println(a[i][j]);
        }
    }
  }
}

public static void main()
  {
  Scanner sc=new Scanner(System.in);
  System.out.println("Enter the row and column");
      int r= sc.nextInt();
      int c= sc.nextInt();

  MatPalin ob = new MatPalin(r, c);

      ob.fill();
      ob.display();
  }
}
```

iii) i)

```
int power(int x, int y)
{
    int p=1;
for (int i=1; i<=y; i++)
  {
     p *= x;
  }
return p;
}
```

ii) The type of cybercrime which private information involves stealing through malicious emails, messages links etc, that trick the user to share sensitive information like passwords, bank details etc is called phishing.